FORMATION « PYTHON EN 24 HEURES » | SÉANCE 04

De l'algo au code : conditions et boucles en Python

Chapitres 7 et 8 du Tome 1

1 h de cours (présentation Beamer) • 1 h de pratique (notebook Jupyter)

HOUNKPE Mawulikplimi Roland et DJABON Ounimborbitibou

Objectifs de la séance

- Traduire SI, SINON SI et SINON en if, elif et else avec la bonne indentation
- Combiner des tests avec and, or et not
- Écrire des boucles while et for, avec range() et les bornes exactes
- Contrôler le flux avec break et continue
- Reconnaître et corriger les erreurs d'indentation

Rappel de la séance 03

Vous savez créer des variables et calculer : il est temps de donner au programme le pouvoir de décider et de répéter.

- Types de base : int, float, str, bool et leurs conversions
- Opérateurs arithmétiques : +, -, *, /, //, %, **
- Priorités : la puissance avant la multiplication avant l'addition
- f-strings : interpolation et formatage {:>10.2f}
- Toute comparaison produit un bool : True ou False

La syntaxe if / elif / else

```
note = 13
if note >= 16:
    print("Très bien")
elif note >= 10:
    print("Passable")
else:
    print("Insuffisant")
```

Les deux points ouvrent le bloc ; le bloc est constitué de tout ce qui est indenté de 4 espaces.

L'indentation est la structure

- En pseudo-code : FIN SI ; en Python : **fin de l'indentation**
- 4 espaces par niveau, jamais de mélange tabulations et espaces
- Tout ce qui est indenté au même niveau appartient au même bloc
- Une indentation incohérente lève ‘IndentationError‘ à l'exécution

Les opérateurs logiques Python

Pseudo-code	Python	Règle
ET	and	vrai si les deux côtés sont vrais
OU	or	vrai si au moins un côté est vrai
NON	not	inverse la valeur logique
=	==	test d'égalité : deux signes !
!=	!=	différent de : identique

Priorité de lecture : not, puis and, puis or. En cas de doute, mettez des parenthèses.

elif contre conditions imbriquées

- Chaîne elif : les cas se lisent en cascade, un seul est choisi
- Imbrication : un if dans un if, utile pour des décisions à deux niveaux
- Au delà de deux niveaux d'imbrication, préférez une chaîne elif
- Règle : la branche la plus probable en premier quand cela compte pour la performance

```
age = 20
carte = True

if age >= 18:
    if carte:
        print("Accès OK")
    else:
        print("Carte manquante")
else:
    print("Trop jeune")
```

La boucle while

```
compteur = 5
while compteur > 0:
    print(compteur)
    compteur = compteur - 1
print("Décollage !")
```

TANT QUE se traduit mot pour mot : condition vérifiée avant chaque tour, mise à jour obligatoire dans le corps.

La boucle for et range()

```
for k in range(1, 11):
    print(k, "fois 7 =", k * 7)

for lettre in "Python":
    print(lettre)
```

for parcourt directement les éléments d'une suite : entiers de range(), caractères d'une str, éléments d'une list.

Les trois visages de range()

Appel	Valeurs produites	Usage
range(5)	0, 1, 2, 3, 4	5 tours, numérotation à partir de 0
range(1, 6)	1, 2, 3, 4, 5	borne de fin exclue
range(10, 0, -1)	10, 9, ... 1	compte à rebours, pas de -1

Règle d'or : la borne de fin n'est jamais atteinte. range(a, b) produit exactement b moins a valeurs.

break et continue

```
for k in range(1, 100):
    if k % 13 == 0:
        print("Premier multiple de 13 :", k)
        break
    if k % 2 == 0:
        continue      # passe au tour suivant
    print("Impair :", k)
```

break quitte la boucle immédiatement ; continue saute le reste du corps et enchaîne le tour suivant.

Simulation d'une devinette

- Nombre secret, essais fournis à l'avance
- Boucle for sur les essais
- Trop petit, trop grand ou trouvé ?
- break dès la victoire
- Nous remplacerons les essais par input() dans vos scripts, jamais dans un notebook

```
secret = 42
essais = [10, 50, 40, 42]

for essai in essais:
    if essai == secret:
        print("Trouvé :", essai)
        break
    elif essai < secret:
        print(essai, "trop petit")
    else:
        print(essai, "trop grand")
```

Attention

Les erreurs d'indentation

Un espace de trop dans une branche change le sens du programme sans message d'alerte logique : l'instruction se rattache alors à un autre bloc. Vérifiez systématiquement les trois zones à risque : après un if, après un for, après un while. L'éditeur compte les espaces pour vous : activez l'affichage des espaces invisibles.

À retenir

L'essentiel de la séance 04

if, elif, else décident ; while et for répètent ; break et continue dirigent la boucle depuis l'intérieur. L'indentation de 4 espaces est la structure même du langage. range(a, b) compte de a à b moins 1 : récitez cette règle jusqu'à ce qu'elle devienne automatique.

Travaux pratiques (1 h) : Conditions et boucles en action

Cinq exercices qui combinent décisions et répétitions, du test le plus simple à la pyramide d'étoiles. Pour chaque exercice, annoncez vous même si vous choisissez while ou for, et pourquoi.

- **Exercice 1 (Basique)** : Signe d'un nombre
- **Exercice 2 (Basique)** : FizzBuzz sur une plage
- **Exercice 3 (Intermédiaire)** : Somme des nombres pairs avec while
- **Exercice 4 (Intermédiaire)** : Devinette avec compteur d'essais
- **Exercice 5 (Défi)** : Pyramide d'étoiles

Ouvrez le notebook « Seance_04_conditions_et_boucles_python.ipynb » pour commencer.

Pour la prochaine séance

Conseil de travail

Séance 05 : les listes et les tuples, puis les compréhensions de listes. Relisez les boucles imbriquées de la séance 02 : elles reviennent pour parcourir les tableaux à deux dimensions.